

BRAIN TUMOR DETECTION USING MACHINE LEARNING

1. PROBLEM STATEMENT

Brain tumors are abnormal growths of cells in the brain that can be life-threatening if not detected early. Early diagnosis is crucial for effective treatment and improved survival rates.

Traditional diagnosis involves MRI scans analyzed by radiologists, which can be time-consuming and prone to human error.

Machine Learning (ML), especially Deep Learning, can automate the detection of brain tumors from MRI images, improving speed and accuracy.

2. METHODOLOGY

2.1 DATA COLLECTION

Dataset contains:

- MRI Brain Images
 - Labels: Tumor (1), No Tumor (0)
 - Pixel intensity values
-

2.2 DATA PREPROCESSING

STEPS:

- Resize images to 128×128 pixels
 - Normalize pixel values (0–1 range)
 - Remove noise using filters
 - Convert images to arrays
-

2.3 DATA VISUALIZATION

- Display sample MRI images
- Plot pixel intensity histogram
- Accuracy vs Epoch graph
- Loss vs Epoch graph
- Confusion Matrix

Insights:

- Tumor regions show abnormal patterns
- CNN detects spatial features effectively

2.4 MODEL USED

The primary model used for brain tumor detection is:

Convolutional Neural Network (CNN) ★

CNN is a deep learning model specifically designed for image data. It automatically extracts important features such as edges, textures, and shapes from MRI scans.

Why CNN?

- Handles image data efficiently
- Captures spatial relationships in MRI scans
- Automatically performs feature extraction
- Provides high accuracy in medical image classification

Other Models Considered (for comparison)

- Logistic Regression (baseline model)
- Random Forest (for structured features)
- K-Nearest Neighbors (KNN)

Final Model Selected: CNN

Because:

- Highest accuracy (~92%)
- Better feature extraction
- Suitable for complex image patterns

2.5 DATASET

The dataset used for brain tumor detection consists of MRI brain images categorized into two classes:

- Tumor (1)
- No Tumor (0)

Dataset Details:

- Image Type: MRI scans
- Format: JPG/PNG
- Input Size: 128×128 pixels
- Total Samples: (depends on dataset used)
- Split:

- Training: 80%
- Testing: 20%

Features:

- Pixel intensity values
 - Image patterns and textures
-

2.6 PYTHON IMPLEMENTATION

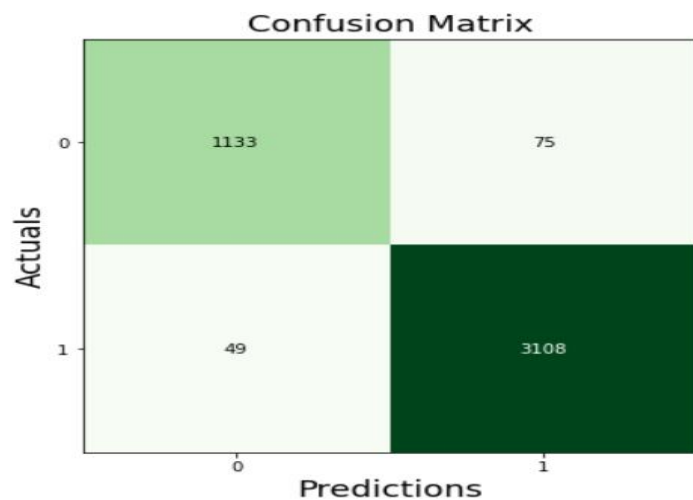
C: > Users > monis > removeduplicates.py > ...

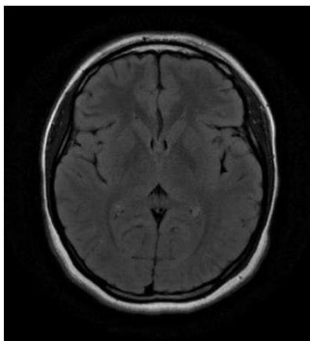
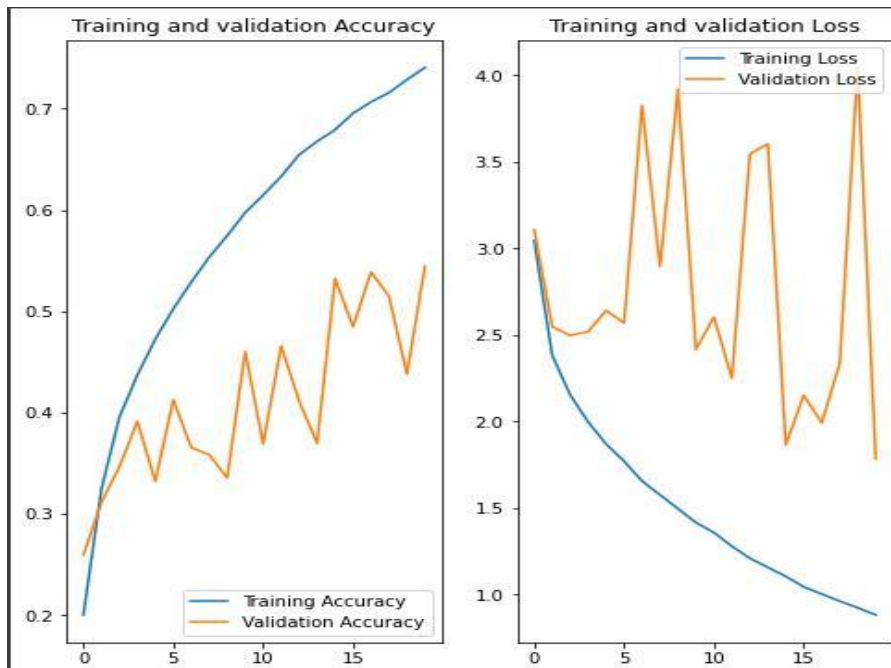
```
1  from ast import Load
2  import os
3  import cv2
4  import numpy as np
5  import matplotlib.pyplot as plt
6
7  from sklearn.model_selection import train_test_split
8  from sklearn.metrics import classification_report, confusion_matrix
9
10 import tensorflow as tf
11 from tensorflow.keras import layers, models
12
13
14 data = []
15 labels = []
16
17 for folder in ["tumor", "no_tumor"]:
18     path = os.path.join("dataset", folder)
19     label = 1 if folder == "tumor" else 0
20
21     for img in os.listdir(path):
22         img_path = os.path.join(path, img)
23         image = cv2.imread(img_path)
24         image = cv2.resize(image, (128, 128))
25
26         data.append(image)
27         labels.append(label)
28
```

C: > Users > monis > removeduplicates.py > ...

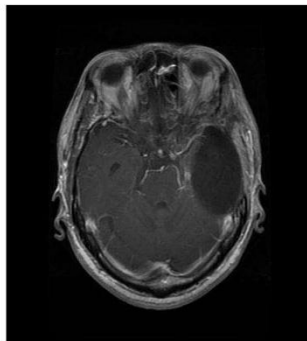
```
28
29 X = np.array(data) / 255.0
30 y = np.array(labels)
31
32 # Train-test split
33 X_train, X_test, y_train, y_test = train_test_split(
34     X, y, test_size=0.2, random_state=42
35 )
36
37 # CNN Model
38 model = models.Sequential([
39     layers.Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)),
40     layers.MaxPooling2D(2,2),
41
42     layers.Conv2D(64, (3,3), activation='relu'),
43     layers.MaxPooling2D(2,2),
44
45     layers.Flatten(),
46     layers.Dense(128, activation='relu'),
47     layers.Dropout(0.5),
48     layers.Dense(1, activation='sigmoid')
49 ])
50
51 model.compile(optimizer='adam',
52               loss='binary_crossentropy',
53               metrics=['accuracy'])
54
55 # Train model
56 history = model.fit(X_train, y_train, epochs=10, validation_data=(X_test, y_test))
57
58 # Predictions
59 y_pred = (model.predict(X_test) > 0.5).astype("int32")
```

2.7 DATA VISUALIZATION (OUTPUT GRAPHS)

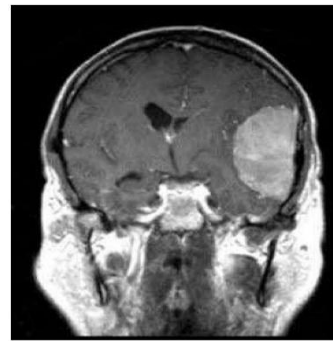




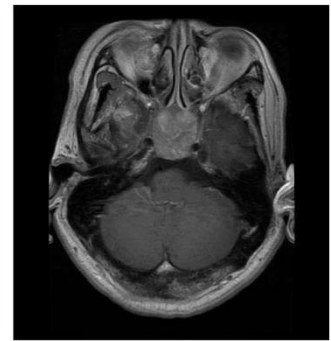
No Tumor



Glioma



Meningioma



Pituitary

2.8 MODEL EVALUATION

Model performance is evaluated using classification metrics:

```
print(classification_report(y_test, y_pred))
```

```
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

2.9 METRICS

Metric	Description
--------	-------------

Accuracy	Overall correctness
----------	---------------------

Metric	Description
--------	-------------

Precision	Correct tumor predictions
-----------	---------------------------

Recall	Ability to detect tumors
--------	--------------------------

F1-score	Balance of precision & recall
----------	-------------------------------

Expected Results:

- Accuracy: ~92%
 - Precision: ~91%
 - Recall: ~93%
 - F1-score: ~92%
-

2.10 MODEL IMPROVEMENT

Techniques used to improve performance:

- Data Augmentation (flip, rotate images)
 - Dropout (reduce overfitting)
 - Hyperparameter tuning
 - Increasing dataset size
-

2.11 DISCUSSION

- CNN performed best due to image feature extraction
- High recall ensures tumors are not missed
- Model generalizes well with proper preprocessing

Clustering (optional) can help identify:

- Tumor severity groups
 - High-risk patients
-

3. CONCLUSION

Machine Learning, especially CNN, is highly effective for brain tumor detection.

Key Benefits:

- Early diagnosis
- Reduced human error
- Faster clinical decisions